

Day 1:

OOP using Java:

Java is an object oriented programming language meant for developing real time business applications.

Application Types:

1. Desktop Application (Stand Alone => Single User)
2. Simple N/W Based Application
3. Web Based Application
4. Mobile Based

Java Features:

1. Simple
2. Object Oriented
3. Platform Independent
4. Architecture Neutral
5. Portable
6. Robust
7. Multithreaded
8. Distributed

Object Oriented Programming:

It is a set of principles or a programming pattern meant for simplifying the development.

Object is an entity having well defined structure.

OOP Principles:

1. Abstraction – Hiding the complexities, Identifying key aspects and ignoring rest
2. Encapsulation – Separation between abstraction and the actual implementation
3. Inheritance – Building new things based upon existing structure
4. Polymorphism – Multiple Forms (Objects responding to same message in different ways)

Getting Started:

Development Environment Setup:

1. JDK (Java Development Kit)
2. Any Code Editor (Preferably some IDE => Eclipse)

Java Compiler:

A Java compiler is used to compile the Java source code.

Java Source File Rules / Concepts:

1. If the class is declared as **public** then the file name must match with the name of the class.
2. The name of the **.class** file depends upon the name of the class rather than the name of the source file.
3. It is possible to define multiple classes in a single source file provided up to maximum only 1 is **public**.

JRE:

JRE stands for Java Runtime Environment.

It is the basic environment required to execute a Java program.

It mainly consists of 3 parts:

1. ClassLoader
2. Byte Code Verifier
3. JVM (Java Virtual Machine)

ClassLoader:

The `.class` file generated by Java Compiler contains a Byte Code.

A byte code is a highly optimized set of instructions and it is a platform neutral code (Not specific to OS) making the language platform independent.

ClassLoader loads the required `.class` files (Loads the byte code).

Byte Code Verifier:

It verifies the byte code whether it is valid and safe or not.

This ensures security.

JVM:

It stands for Java Virtual Machine.

It converts platform neutral code into machine specific code.

It acts as an interpreter.

It is highly platform dependent.

Data Types:

Java is a strictly typed language.

Every variable, expression or even assignment is checked for type compatibility.

There are 2 types of variables: Primitives and Non-Primitives

Primitives:

They are expressed using several data types.

There are 8 data types:

1. Numbers
 - a. Integers (byte 1, short 2, int 4, long 8)
 - b. Fractions (float 4, double 8)

Characters (char 2)

Booleans (boolean 1 bit)

Non-Primitives:

They are expressed using references and therefore they are also called as Reference Types:

1. Strings
2. Arrays
3. Other Objects

Java Data Types differences as compared to C and C++:

1. Does not have *unsigned* integers
2. Conditional statements must evaluate to a boolean expression.
3. *char* occupies 2 bytes because Java follows UNICODE (0 to 65535) character set and not ASCII (0 to 255) .
This is because Java is meant for Internationalization (i18n).

Working with Classes:

A class is a template used to decide a structure for an object.

Once a class is defined, it can be used to create several objects.

Creating Objects:

In order to create objects, Java provides `new` keyword.

Working with Methods:

A class can have one or multiple method definitions.

Methods are especially used to implement encapsulation through data hiding.

This is done using `private` access modifier.

Accessor and Mutator Methods:

Accessor methods are used to retrieve values of the individual fields whereas mutator methods are used to modify values of the individual fields.

They are also called as Getters and Setters.

Ideally they have to follow naming conventions.

Method Overloading:

When 2 or multiple methods have same name but different signatures, then they are called as overloaded methods.

Signatures are considered to be different if one of the following is satisfied:

1. No. of parameters is different
2. Types of parameters are different
3. Sequence of parameters is different

Return types are not taken into consideration.

If method with matching signature is not found, compiler gives error.

Unlike C and C++, Java does not go for garbage values; rather it goes for default values.

These default values depend upon type of the variables e.g. Integers are initialized to 0, Booleans : false, Non-Primitives : null and so on.

In order to customize initialization, Java provides a special member known as Constructor.

It has same name as that of class name, it does not have return type and it gets invoked implicitly as soon as object is created.

Like methods, constructors can also be overloaded.

Understanding **this**:

Every method or a constructor defined within a class has a hidden parameter known as **this** reference.

It always refers to the current object that is in progress.

It is required to be used explicitly when local variable names conflict with instance variable names.

It is used to resolve the scope because priority is always given to the local variable.

Day 2:

Static Members:

Whatever definitions are done within the class are called as members of a class e.g. variables, methods, constructors and so on.

These members can be of 2 types:

1. Non-Static
2. Static

Non-Static Members:

These are the characteristics associated with a particular instance or an object.

Static Members:

These are the characteristics associated with a class and not an object.

There are 2 types of static members: variables and methods.

Constructors cannot be declared as *static*.

Static Variables:

When a variable is declared as *static*, a single copy is created of that variable irrespective of the number of objects.

Static Methods:

When a method is defined as *static*, it can be invoked without an object.

Non-Static Vs Static Methods

Non Static methods must be invoked using some object reference whereas Static methods can be invoked without an object reference; rather they are ideally to be invoked using a class name.

Non-Static methods can access non-static as well as static members whereas Static methods can access only static members.

Static methods cannot make use of *this*.

Static Initialization Block:

It is an arbitrary block of code that gets executed when the class is loaded.

It can be used to perform operations during class loading e.g. initializing static variables.

Memory Mapping:

In the context of Java, memory is mainly divided into 2 sections:

Stack and Heap.

When an object is created, its reference is stored on stack whereas the actual object is stored on heap.

An object may have one or many references whereas a reference can refer to only one object at a time.

Garbage Collection:

Java Runtime System implies a special mechanism known as Garbage Collection.

It internally keeps track of the number of references referring to an object i.e. reference count.

As soon as the reference count becomes 0, the object is marked as UNUSED and it becomes eligible for garbage collection.

There is a background activity known as Garbage Collection that runs periodically and identifies the objects which are marked as UNUSED and further it starts de-allocating memory for those objects.

It is possible to request garbage collector to execute via *System.gc()*.

Parameter Passing:

A method may accept parameters either in the form of primitives or non-primitives.

In case of primitives, parameters are always passed by VALUE.

A copy of a variable is created on stack and it gets passed further.

In case of non-primitives, parameters are always passed by REFERENCE.

A copy of a reference and not the object is created on stack and it gets passed further.

Arrays:

An array is a collection of similar typed elements stored at contiguous memory locations.

Arrays can be created by 2 ways:

1. Only Declaration => `int[ ] numbers = new int[5];`
2. Declaration and Initialization at same time
`int[ ] numbers = {56,34,12};`

Every array itself is an object and therefore the entire array is stored on heap whereas its reference is stored on stack.

All array objects have a common property: `length`.

It is possible to create an array of user defined types as well.

It is possible to pass an array as a parameter or even return it back.

It is also possible to declare a dynamic array.

Java Relationships:

In Java programming model, relationships are of 2 types:

1. HAS-A
2. IS-A

HAS-A:

It is implemented using Containment.

When an object contains another object or several objects then it is called as Containment.

There are 2 forms of containment:

1. Aggregation (Weak Dependency)
2. Composition (Strong Dependency)

IS-A:

It is implemented using Inheritance.

When a class inherits another class then it is called as inheritance.

Java provides `extends` keyword to achieve this.

Java supports 3 types of inheritances:

1. Simple
2. Multilevel
3. Hierarchical

Inheritance in Java is always *public*.

Constructors in inherited classes:

Constructors in inherited classes are invoked from bottom to top whereas they are executed from top to bottom.

In case of no-argument constructor, Java adds a statement by default: `super()` where *super* is a keyword used to pass on the control to the super class constructor.

Especially it is required in case of parameterized constructors because there can be multiple parameterized constructors available within a single class.

If omitted, it defaults to *super()* which invokes no-argument constructor of base class.

It must be the first statement.

Method Overriding:

When methods have same name, same signature and same return type defined within inherited classes, then they are called as overridden methods.

Overriding is used to modify the behavior of the base class method to suit the new requirement.

Polymorphism:

It involves methods with same name but different implementations.

It is of 2 types:

1. Static (Compile Time) => Method Overloading
2. Dynamic (Runtime) => Method Overriding

In method overloading, methods calls are resolved at compile time.

If compiler cannot find the corresponding method with matching signature, it reports an error.

In case of method overriding, method calls are resolved based upon dynamic type of the reference.

Every reference has 2 types: Static and Dynamic

Since method selection happens based upon dynamic type, method overriding is considered as dynamic polymorphism.

Method Overriding Use Case:

Method Overriding can be used in following contexts:

1. Replacement
2. Extension

In case of replacement, the derived class method completely replaces the functionality of a base class method.

In case of extension, the derived class method extends the functionality of a base class method.

This is done using *super* keyword.

E.g.

`super.methodName()` => Invokes method from super class.

Day 3:

Abstract Classes: (Reference: Greeting.java)

When a class provides a method that is not to be defined but only declared, Java enforces that method to be explicitly declared as **abstract**.

Any class that contains at least one method declared as **abstract**, must be declared as **abstract**.

When a class is declared as **abstract**, it cannot be instantiated.

It can be used further by creating sub-classes.

The subclass must implement all the **abstract** methods of the super class or else must be declared itself as **abstract**.

Abstract classes may contain concrete methods also.

The **abstract** modifier cannot be used for variables, constructors and static methods.

Working with **final**: (Reference: Greeting.java)

A modifier used to make variables immutable (Value cannot be changed).

If **final** is used in case of non-primitives, the reference must stay the same and not the object.

It is also possible to define methods as **final**.

If so, such methods cannot be overridden.

If a method is **private**, it is implicitly **final**.

It is also possible to define a class as **final**.

If so, it cannot be inherited.

All the methods of a **final** class implicitly become **final**.

Interface:

Interface is a collection of **abstract** methods and possibly **final** variables.

It is used to declare methods where implementation is not available.

An interface is declared using **interface** keyword.

Like abstract class, interface cannot be instantiated.

It is to be used further by creating its implementation class.

Java provides **implements** keyword to achieve this.

Why Interface:

Interface is used mainly to achieve following:

1. Expanding the scope of polymorphism
2. Achieving multiple inheritance in terms of methods.

Interface Rules:

1. Methods of a **public** interface are by default **public** and **abstract**.
2. Variables of a **public** interface are by default **public**, **static** and **final**.
3. If a class implements an interface, that class either has to implement all the methods of that interface or else must be declared as **abstract**.
4. A class can implement multiple interfaces.
5. An interface can be implemented by multiple classes.
6. An interface can extend multiple interfaces.

Abstract Class Vs Interface

1. Abstract classes are used to achieve polymorphism for co-related classes whereas interfaces are used to achieve the same for non-co-related classes.
2. A class can extend only one abstract class but can implement multiple interfaces.

Inner Classes:

When a class is defined within another class then it is called as an inner class.

Inner classes are especially used when complex logic is to be isolated but handled locally.

This reduces the overhead of sending parameters or data across method calls.

Inner classes are also used very frequently in GUI programming model.

Types of Inner Classes:

1. Static Inner Class
2. Nested Class
3. Local Class
4. Anonymous

Static Inner Class:

When an inner class is declared with **static** modifier, then it is called as a static inner class.

It can be instantiated without an object of an outer class however its name should be referred through the name of an outer class.

It can access only **static** members of an outer class.

Nested Class:

When an inner class is declared without **static** modifier, then it is called as a nested class.

It must be instantiated though an object of an outer class.

It can access **static** as well as non-static members of an outer class.

Local Inner Class:

When an inner class is defined locally within a specific block then it is called as a local inner class.

It is loaded and instantiated at same time.

Instantiation happens for every invocation of that particular block.

Anonymous Inner Class:

It is an inner class defined without name.

It is especially used in the context of either abstract classes or interfaces.

It is used to implement the functionality which is a rare requirement.

Java compiler creates a *.class* file for every building block irrespective of its type.

In case of inner classes it uses '\$' notation.

E.g. Outer\$Inner.class

Package:

A package is a collection of one or multiple building blocks like classes, interfaces, annotations, enums, records and so on.

It is used to isolate library from other libraries.

It is also used to reduce naming conflicts.

A package is declared using *package* keyword.

When 2 classes reside within same package, they can access each other directly irrespective of whether they are declared as **public** or not.

When 2 classes reside within different packages, in order to make them accessible from outside the package, they need to be exported.

It is done using **public** access modifier.

Once the class is exported, other classes can import it using **import** statement.

For every building block e.g. class, interface etc, Java creates a `.class` file whereas for every package, it creates a folder with same name.

Sub Package:

It is possible to create a package inside another package which is called as a sub package.

Java uses `'.'` notation to achieve this.

In order to import classes from sub package, its name must be referred through the name of super package.

E.g.

```
import desert.brownie.WalnutBrownie;
```

It is also possible to import all **public** declarations using `'*'`.

E.g.

```
import food_item.*;
```

Default Package:

When a class is defined without any explicit package, by default it goes into some package that is referred as Default Package.

It is not recommended.

Access Modifiers:

Java provides 4 access modifiers

1. **public** – Accessible from anywhere
2. **private** – Accessible only within the class
3. **protected** – Accessible within the entire package as well as outside the package if the class is a sub class.
4. **DEFAULT** – Accessible only within the entire package.

Exploring Java's Built-In Packages:

Java provides wide range of built-in classes, interfaces, annotations and so on.

All these blocks are defined within some package as it is a standard practice.

At the base, Java provides a package called as **java.lang**.

It is the package imported by default.

This makes all the public declarations accessible to the developer by default.

Package **java.lang** provides several classes e.g. String, System and so on.

However, the most significant class is **Object**.

It is the class from which all classes are inherited by default.

The **Object** class provides a basic foundation with the help of several methods.

1. *toString()*
2. *equals()*
3. *finalize()*
4. *clone()*
5. *getClass()*
6. *hashCode()*
7. *wait()*
8. *notify()*
9. *notifyAll()*

`public String toString():`

Used to convert an object into String.

When any object reference is passed as a parameter to *System.out.println()* statement, JRE identifies the dynamic type of that reference and searches for *toString()* method in the respective implementation class.

If found, invokes that method otherwise invokes the method from the base class i.e. *Object* if not inherited explicitly.

The *toString()* from *Object* class returns address of an object.

In order to modify its working, it needs to be overridden.

A call to *toString()* can be either implicit or explicit.

`public boolean equals(Object):`

Used to compares objects for equality.

If objects are compared for equality using '==' operator, it results into *true* if their corresponding references refer to same object; otherwise *false*.

This can be overcome using *equals()* method.

The default implementation of *equals()* method present in *Object* class works exactly same as that of '==' operator.

In order to compare objects for equality based upon their data values, the method needs to be overridden.

Wrapper Classes:

These are the classes used to represent primitives in the form of objects.

For every primitive type, Java provides a corresponding wrapper class.

E.g.

byte => Byte
short => Short
int => Integer
long => Long
float => Float
double => Double
char => Character
boolean => Boolean

All wrapper classes are **final**.

Usage:

```
int x = 10;  
Integer in = new Integer(x);  
int y = in.intValue();
```

Since Java version 5, a special feature has been introduced:

Autoboxing and Unboxing

`int x = 10;`

`Integer in = x;` => Autoboxing

`int y = in;` => Unboxing

Wrapper classes can also be used to convert qualified strings into the respective primitive types.

`String myAge = "57";`

`int age = Integer.parseInt(myAge);`

`String myWeight = "75.83";`

`float weight = Float.parseFloat(myWeight);`

String: (Reference: Greeting.java)

Java provides a class named as *String* that maintains an array of characters.

In Java, strings are immutable.

Any attempt to modify the content results into new object creation.

Java provides a constant pool for String objects.

String objects can be created without *new* keyword. If so, constant pool gets used.

If they are created using *new* keyword, constant pool does not get used.

StringBuilder:

Unlike String, StringBuilder follows mutable sequence of characters.

This allows to optimize the performance when heavy string manipulation is required.

It provides several overloaded *append()* methods.

Day 4:

Exception Handling:

Exception Handling is an object oriented way of handling errors that occur during program's execution. Problem solving code is separated from error handling code that adds simplicity.

In order to handle exceptions, Java mainly provides 2 keywords: **try** and **catch**.

Both of these are to be used with the help of blocks.

The *try* block is used to enclose the statements which are probable to fire an exception.

The *catch* block is used to handle the exception. It acts as an exception handler.

Exceptions are raised by JRE when an abnormal situation is encountered.

JRE identifies the abnormal situation and maps it to the corresponding exception specific class.

E.g. Index out of bounds => `ArrayIndexOutOfBoundsException`

After that JRE creates an object of the exception specific class and searches for exception handler i.e. *catch* block.

Once found, invokes that block.

Types of Exception:

There are 2 types of exceptions: Unchecked and Checked

Unchecked Exceptions:

These are the exceptions for which compiler does not enforce to handle them.

Checked Exceptions:

These are the exceptions for which compiler enforces to handle them.

Exceptions hierarchy provides a topmost class *Throwable* from *java.lang* package.

It is derived by 2 subclasses: *Error* and *Exception*

Error:

It indicates resource exhaustion in JVM.

It is rare but fatal.

It is not under the control of a developer.

It has several subclasses e.g. *StackOverflowError*, *OutOfMemoryError*.

Exception:

It is a runtime error that occurs due to programming mistakes.

It is frequent and not fatal.

It is under the control of a developer.

Handling Multiple Exceptions:

A single block of code may fire different types of exceptions.

Still it is possible to handle them using multiple *catch* blocks.

Handling Default Exception:

Sometimes a block of code may fire some exception other than those which have been already handled.

In this case, it is necessary to handle that default exception otherwise it results into system message.

This is done by providing a *catch* block catching any type of exception i.e. *Exception*.

Working with **finally**:

Java provides a *finally* block that gives a guarantee about statement's execution.

Irrespective of whether an exception is raised or not, *finally* block always executes.

The *finally* block creates an impact especially when the methods having a return type other than *void*.

Working with *try-catch-finally*:

1. These blocks must be used in the same order as they appear.
2. These blocks must appear one after the other without any statement in between.
3. The *try* block must be accompanied with either *catch* or *finally* or both.
4. Blocks *catch* and *finally* cannot be defined without *try*.
5. It is possible to have nesting of the blocks.

Java provides 2 more keywords in the context of exception handling: **throws** and **throw**.

Working with *throws*:

Sometimes a method or constructor statements are capable of throwing some exception but not willing to handle it.

This can be achieved by expanding their definition using *throws*.

E.g.

```
public void readFile(String filename) throws FileNotFoundException{  
    //Code to open the file and read it  
}
```

In the above fragment, method *readFile()* is capable of raising *FileNotFoundException* but not willing to handle it; rather it wants its caller to handle it. This is expressed using *throws*.

This rule is applicable to CHECKED exceptions only.

In method overriding, the overridden method can widen the scope but cannot narrow it. (Ref: ThrowsMain.java).

Working with *throw*:

Sometimes it is necessary to throw an exception based upon some condition.

In this case, instead of waiting for JRE to throw the exception, it is possible to throw it explicitly using *throw* keyword.

User Defined Exception:

These are the exceptions created by developers as per the domain specific requirement.

It is done by defining a class that is called as an exception specific class.

In order to configure the class as an exception specific class, it needs to be inherited from one of the classes available in exception hierarchy.

Usually, for UNCHECKED exceptions, *RuntimeException* is used as a base class and for CHECKED exceptions; *Exception* is used as a base class.

Multithreading:

Multitasking:

It involves executing multiple tasks at same time.

It has 2 forms:

1. Multiprocessing
2. Multithreading

Multiprocessing:

It involves executing multiple processes at same time.

Program is a set of instructions whereas every running instance of a program is called as a Process.

CPU is shared between 2 processes.

OS implements context switching algorithm to manage multi-processing.

The entire context i.e. variables, global variables, objects and so on, is stored separately.

Multithreading:

It involves executing multiple threads at same time.

A thread is an entity or a task within a process.

It defines the path of execution.

Every Java application has a main thread that begins at the start of the *main()* method and ends at its end.

Multi-Processing

Vs

Multi-Threading

1. In multiprocessing, process is the smallest unit of work whereas in multithreading, thread is the smallest unit of work.
2. In multiprocessing, the entire context is stored separately whereas in multithreading, since multiple threads live within the same process, may share the data.
3. Process is a heavy-weight entity whereas thread is a light-weight entity.
4. Inter-process communication is a heavy-weight activity whereas inter-thread communication is a light-weight activity.

Implementing Multithreading:

The task that is to be performed by a thread, its relevant logic is to be written within a Java class that is referred as thread implementation class.

Java provides 2 APIs to implement multithreading:

1. *Thread* (Class)
2. *Runnable* (Interface)

The thread implementation class has to either extends *Thread* class or implement *Runnable* interface.

Once the class is defined, the further logic for the thread is to be defined using a method: *public void run()*.

Thread Life Cycle:

The life cycle of thread mainly consists of 5 states:

1. BORN
2. READY
3. RUNNING
4. BLOCKED
5. DEAD

BORN:

When a thread is newly created, it is said to be in BORN state.

READY:

When the thread is eligible to execute, it is said to be in READY state.

RUNNING:

When the thread's execution begins, it is said to have entered into the RUNNING state.

When this state is acquired, *run()* method gets invoked.

BLOCKED:

At any time, a currently running thread can be sent into some blocked state.

There are 4 possible BLOCKED states:

1. SLEEPING
2. SUSPENDED
3. BLOCKED for IO Request
4. WAITING

DEAD:

When the thread completes its task, it is said to have entered into DEAD state.

Day 5:

Thread Priorities:

Threads have priorities ranging between 1 to 10 where 1 is minimum and 10 is maximum.

The *Thread* class provides a method: *setPriority(int)*. By default it is 5.

Thread Scheduling:

In the context of multithreading, OS implements scheduling algorithm i.e. scheduler used to schedule the threads.

Therefore the behavior of multithreaded applications is platform dependent.

There are mainly 2 types of scheduling:

1. Priority Based: Threads are scheduled as per their priority.
2. Preempting: Apart from priority based scheduling, the higher priority thread may preempt lower priority thread.

Thread class methods:

1. *start()*
2. *stop()*
3. *sleep()*
4. *suspend()*
5. *resume()*
6. *currentThread()*
7. *yield()*
8. *join()*

start():

It requests OS for the creation of the thread.

It brings a thread into READY state.

It internally gives a call to *run()* method.

stop() :

Terminates the thread abruptly and sends it into DEAD state.

This is not much recommended and therefore this method is now deprecated (outdated).

sleep():

A *static* method that causes currently running thread to enter into SLEEPING state.

The thread leaves SLEEPING state and goes back to the READY state as soon as the sleep time interval is over.

suspend():

Sends the thread into SUSPENDED state.

resume():

Brings the suspended thread back into READY state.

Since this activity is implementation dependent, both these methods are deprecated.

currentThread():

A *static* method used to obtain a reference to the currently running thread.

yield():

A *static* method used to give up the control to some other thread.

Causes the currently running thread to leave the RUNNING state and go back to READY state.

Probably, scheduler schedules the thread having priority at least as high as that of the thread that has been yielded.

`join()`:

It causes a parent thread to wait until the termination of the child thread.

Thread Group:

It is possible to create a group of threads.

This allows to perform operations collectively on 2 or multiple threads involved in that group.

Java provides a relevant class: *ThreadGroup* to accomplish this.

Daemon Threads:

These are service threads or background threads as they run in a background to perform specific task.

E.g. Garbage Collector is a daemon thread as it runs in the background.

In order to launch a thread as a Daemon thread, *Thread* class provides *setDaemon(boolean)* method.

When there are only DAEMON threads left and all non-daemon threads are terminated, JVM terminates.

Synchronization:

Since multiple threads run within a same process, they may share same data.

When this happens so, sometimes it is necessary to ensure that the data manipulation is done by only 1 thread at a time.

That means, ideally the object must be modified by only 1 thread at a time.

This technique is known as synchronization.

If not handled properly, both the threads fall into RACE condition and that may lead to the problem of data inconsistency.

The key to synchronization is the monitor.

A monitor is a mutually exclusive lock.

Only one thread can own a monitor at a time i.e. the thread that owns a monitor is said to have acquired a lock on the object.

All other threads have to wait for the same lock and they can acquire the lock only when the current thread that has acquired a lock, releases it.

Implementing Synchronization:

In order to implement synchronization, Java provides **synchronized** keyword.

It can be used through 2 options:

1. Using Method
2. Using Block

Synchronized Method:

When a currently running thread invokes a method on an object and if the method is declared as *synchronized*, the thread acquires a lock on that object.

This lock remains as long as control stays within the method and it gets released when the control exits from the method.

Synchronized Block:

Sometimes, the object that is being shared by multiple threads is created of some legacy class where synchronization is not handled.

In this case, in order to handle synchronization, a synchronized block is an appropriate option.

Thread-Safe Class:

When a class is defined, developer may define the methods of that class as *synchronized* keeping future requirements in mind.

If so, such a class is known as a Thread-Safe class.

E.g.

StringBuffer (Since Java 1) from *java.lang* package is a thread-safe class whereas *StringBuilder* (Since Java 5) is not a thread-safe class.

Inter-Thread Communication:

When 2 threads communicate with each other, it is called as inter-thread communication.

In order to achieve inter-thread communication, both the threads must share same object.

When same object is being shared, synchronization is also required.

In order to implement inter-thread communication, 3 methods are used:

1. *wait()*
2. *notify()*
3. *notifyAll()*

All these methods belong to *java.lang.Object* class and all of them are *final*.

All these methods must be invoked within a synchronized context otherwise *IllegalMonitorStateException* is raised.

wait():

It releases the lock explicitly and sends the currently running thread into WAITING state.

notify():

When the other thread enters into the locked monitor and invokes *notify()*, sends the notification to the thread that is in WAITING state.

The WAITING thread leaves that state and goes into READY state.

notifyAll():

Performs same operation as that of *notify()* but notifies all WAITING threads.

IO Programming:

Every program needs some input and delivers output.

In order to fetch the input, some SOURCE is required whereas for output some DESTINATION is required.

There are several sources possible e.g. Input Device (Keyboard), File, Socket etc.

There are several destinations possible e.g. Output Device (Console), File, Socket etc.

There are 2 basic operations involved: READ and WRITE.

To perform READ operation some SOURCE is used and for WRITE operation, some DESTINATION is used.

Java provides extensive support for handling IO operations through a library (API) that belongs to *java.io* package.

At the base, it provides 2 classes:

InputStream and *OutputStream*

To perform any operation whether READ or WRITE, it is necessary to open a stream.

Stream depicts the flow of bytes (data) and for READ operation, it is handled by *InputStream* and for WRITE operation, it is handled by *OutputStream*.

Both these classes are *abstract* classes and they have to be used through their corresponding subclasses.

E.g.

To perform READ and WRITE operations on Files, Java provides 2 classes:

FileInputStream and *FileOutputStream*

Reading Data from File:

In order to read data from file, Java provides *FileInputStream* class.

The object of *FileInputStream* maintains a file pointer and by default it starts pointing to BOF (Beginning of File) position.

In order to shift the pointer (cursor) one step ahead, *read()* method is used.

It reads one character at a time returns its value and further shifts the cursor one step ahead.

When EOF (End of File) position is encountered, it returns -1.

Working with **try-with-resources**:

It is a feature introduced by Java version 7.

It allows to close the resource automatically once its job is finished.

Java version 7 provides an interface from *java.lang* package: **AutoCloseable**.

Any type which falls under *AutoCloseable*, the resource of that type can be opened using *try-with-resources* so that it gets closed of its own.

Writing Data into File:

In order to write data into the file, Java provides *FileOutputStream* class.

While writing data into the file, the file need not to be present.

If it is not present, it gets created.

To perform WRITE operation, *write()* method is used.

By default, the file gets opened in WRITE mode.

If the file is not present, it gets created otherwise it gets overwritten.

In order to retain previous data, it must be opened in APPEND mode.

This is done by sending 2nd parameter *true* to the constructor of *FileOutputStream* class.

Working with *InputStream*:

It provides a basic foundation to read the contents and since it is an *abstract* class, it needs to be used through its several subclasses.

E.g. *FileInputStream* is used to read the data from file.

There are several other subclasses e.g. *FilterInputStream*.

FilterInputStream:

It is used to convert raw bytes of information into the desired form.

It has to be used through its several subclasses.

E.g.

1. *LineNumberInputStream*
2. *SequenceInputStream*
3. *BufferedInputStream*
4. *DataInputStream*

LineNumberInputStream:

Used to read the contents based upon line number.

SequenceInputStream:

It is used to place sources in a sequential manner so that the READ operation is simplified.

BufferedInputStream:

When a *read()* method is invoked, every time it makes a disk call.

As the number of disk calls goes on increasing, it slows down the performance.

To optimize the performance, it is necessary to reduce the number of disk calls which is possible using *BufferedInputStream*.

It is used to apply a buffering model which reduces the number of disk calls.

DataInputStream:

It is used to read Java's primitive data types.

It provides relevant readXXX() methods to achieve this.

E.g.

readInt() => Reads int value

readLong => Reads long value

Working with *File*:

Java provides a class *File* to perform file specific operations.

Objects of type *File* do not actually open any file.

It provides several method to perform file specific operations:

1. *exists()*
2. *isFile()*
3. *isDirectory()*
4. *createNewFile()*
5. *mkdir()*

Day 6:

Working with *RandomAccessFile*:

It is used to place file pointer anywhere in the file.

It is not a subclass of any *InputStream* or *OutputStream*.

Therefore, it can be used to perform both the operations: READ and WRITE.

It supports random access through *seek()* method.

Stream Categories:

Since Java is a language meant for Internationalization (i18n), its *char* data type occupies 2 bytes of memory.

Java follows UNICODE character set and not ASCII.

If the file contains UNICODE characters (Characters from languages other than English), there must be a provision to read those characters.

Therefore Java provides relevant stream category.

There are 2 categories of Streams:

1. Byte Streams
2. Character Streams

Byte Streams:

These streams work upon 8-bit data and they are expressed using *abstract* base classes: *InputStream* and *OutputStream*.

They are further used by their corresponding subclasses e.g. *FileInputStream* and *FileOutputStream*.

Character Streams:

These streams work upon 16-bit data and they are expressed using *abstract* base classes: *Reader* and *Writer*.

They are further used by their corresponding subclasses e.g. *FileReader* and *FileWriter*.

The most significant class among character streams is *BufferedReader*.

It provides a *readLine()* method that is used to read the contents line by line.

Accepting User Input:

Accepting input from user is a very common requirement. In GUI enabled applications it is done using various UI components like text box, combo box, check box, radio button etc.

In case of console based applications (Non GUI), user needs to be prompted for the input and further it is provided using a keyboard i.e. Input Device.

In order to read the data entered through Input Device i.e. keyboard, it is necessary to use an *InputStream* that is associated with the Input Device.

This is made possible using a class: *System* from *java.lang* package.

It provides several *public*, *static*, *final* members out of which 2 are frequently used: **out** and **in**.

In order to accept user input, the member **in** which is of type *java.io.InputStream* is to be used.

E.g.

InputStream keyboard = *System.in*

To accept user input, *BufferedReader* can be used but since its *readLine()* method returns a String, it is necessary to convert the string to the desired type as per the requirement.

This can be simplified using another class: *Scanner* from *java.util* package. It is introduced by Java version 5.

It provides several utility methods:
nextInt(), *nextFloat()*, *nextDouble()* and so on.

Serialization:

The process of storing state of an object to some permanent persistent store is known as Serialization.

This allows to use the data again and again.

De-serialization:

The process of obtaining the state back from the persistent store is known as de-serialization.

In order to implement serialization and de-serialization, Java provides an API from *java.io* package.

The API mainly provides:

1. *Serializable*
2. *ObjectOutputStream*
3. *ObjectInputStream*

Serializable:

It is an interface required to be implemented by a class of which an object is to be serialized.

If not implemented, it results into an exception: *NotSerializableException*.

It does not provide any method; rather it is a marker interface.

ObjectOutputStream:

It is a class used to actually serialize the object.

It uses some *OutputStream* as a destination for serialization.

It provides *void writeObject(Object)* method to perform serialization.

ObjectInputStream:

It is a class used to actually de-serialize the object.

It uses some *InputStream* as a source for de-serialization.

It provides *Object readObject()* method to perform de-serialization.

If a class declares any *static* variable, its state does not get serialized at the time of serialization.

During de-serialization, it gets initialized to default value.

The state of non-static variables always gets serialized by default.

To prevent serialization of non-static variables, Java provides *transient* keyword.

Transient variables are also initialized to default value during de-serialization.

It is also possible to perform encryption and decryption during serialization and de-serialization respectively.

This is especially required while working with sensitive information e.g. user credentials, secret codes etc.

Java provides an interface: *Externalizable* which is extended from *Serializable*.

Unlike *Serializable*, it is not a marker interface.

It provides 2 methods: *writeExternal()* and *readExternal()* to handle encryption and decryption respectively.

Both these methods are callback methods (get called implicitly) and get called during serialization and de-serialization respectively.

Collections Framework:

Collection is a data structure meant for storing multiple values at one place.

Even though array can be used to store multiple values at one place, it is similar to collection but not a part of collections framework as it is different from Java's collections.

Array Vs Collection:

1. Array is a collection holding similar typed elements whereas a collection can hold elements of dissimilar types.
2. An array can hold elements of primitive as well as non-primitive types whereas a collection can hold elements only of non-primitive types.
3. An array has a fixed dimension whereas a collection grows dynamically.

Java provides extensive support for handling collections through the package: `java.util`.

It provides wide range of classes and interfaces to handle collection oriented functionalities.

Legacy Classes:

1. *Stack*
2. *Vector*
3. *Dictionary*
4. *Hashtable*
5. *Properties*

Stack:

It is a collection that follows LIFO (Last In First Out) algorithm.

It provides several methods out of which 3 are used frequently:

1. *push()*: Used to push the element onto the stack
2. *pop()*: Used to remove the element from top.
3. *peek()*: Returns the element from top but does not remove it.

Every collection in Java has 2 properties:

1. *size* : Indicates the number of elements present in the collection
2. *capacity* : Indicates the number of elements the collection can accommodate.

Non-Legacy Classes:

Java started providing further classes regarding collection framework for better performance, enhancement etc.

E.g.

ArrayList and *LinkedList*

ArrayList provides almost same functionality as that of *Vector* however it is different from *Vector*.

Vector is thread-safe and *ArrayList* is not.

LinkedList:

It works almost same as that of other collections e.g. *Vector*, *ArrayList* etc; however it provides additional functionality that allows to perform operations from both ends.

It provides relevant methods to achieve this.

Collections Framework Design:

The collections framework designed by Java consists of wide range of predefined interfaces.

Each interface has its own contract and the same is followed by its corresponding implementation class.

At the base, Java provides an interface: *Collection* and it is derived by 2 interfaces: *List* and *Set*.

List:

It has following contract:

1. It is ordered
2. It permits duplicate values

It maintains order of the element using an index and like array, indexing starts from 0.

It has several implementation classes:

1. *Stack*
2. *Vector*
3. *ArrayList*
4. *LinkedList* (Apart from *List*, it implements *Deque*)

Generics:

Java version 5 provides a feature known as Generics.

It allows to implement the functionality where the type is unknown.

Typically it is used by defining an interface that is known as generic interface.

Java uses '< >' notation for generics.

The actual type is specified by the corresponding implementation class.

The generics feature is used in the context of collections framework to make collections type-safe.

When a collection is declared to hold objects of a specific type then that collection is called as type-safe collection.

E.g.

```
List<String> cities = new ArrayList<>();
```

The above statement instructs compiler that the collection named as *cities* can accept only *String* objects.

Any attempt to add objects other than *String* results into compilation error.

Benefits:

1. Early type checking is possible
2. No explicit cast is needed.

Working with wildcards:

It is possible to use wildcards in the context of type-safe collections.

They are especially used while working with inherited classes or even interfaces.

In order to use wildcards, Java provides '?' notation.

Further, there are 2 keywords in use:

1. *extends*
2. *super*

The *extends* is used to indicate upper bound whereas *super* is used to indicate lower bound.

E.g.

Assumption:

Classes *PermanentEmployee* and *ContractEmployee* are subclasses of *Employee*.

```
List<? extends Employee> employees = new ArrayList<>();
```

In this case the collection: *employees* can hold objects of either *Employee* class or its subclasses e.g. *PermanentEmployee* and *ContractEmployee*.

```
List<? super Employee> employees = new ArrayList<>();
```

In this case the collection: *employees* can hold objects of either *Employee* class or its super classes.

Day 7:

Working with *Set*:

This is another type of collection that has following contract:

1. It is unordered.
2. It does not accept duplicate values.

It has a single implementation class: *HashSet*.

Unlike *List*, *Set* does not use index for storing the elements as it is unordered; rather it uses hashing algorithm to calculate hash value and the same is used for storing the element.

The hashing algorithm is to be provided using the *hashCode()* method from *java.lang.Object* class. It is to be overridden in the respective subclass.

While adding objects in a hash based collection, the corresponding implementation class of the object must override *equals()* and *hashCode()* methods.

The *equals()* method is meant for handling the uniqueness whereas *hashCode()* method is meant for providing hashing algorithm.

Working with *hashCode()*:

The *Object* class provides *hashCode()* method that is used to implement hashing algorithm.

A hashing algorithm is an algorithm that calculates an integer value based upon the data present inside the object. This value is also known as hash value or even hash code.

This hashing algorithm is any customized algorithm provided by the class owner.

The hash value calculated from hashing algorithm is used for storing the element in a hash based collection.

When an element is getting added in the hash based collection, the hash value of that element is calculated based upon the hashing algorithm.

For every new hash value, a new memory is allocated for a container called as hash bucket and the relevant element gets stored into the respective hash bucket.

When 2 objects are equal, their hash values are always equal.
However, when 2 objects are unequal, still they may have same hash values.

If the hash value of the new element is the older one then the hash based collection starts comparing the new element with the existing elements available in the corresponding bucket using *equals()* method.

Working with *Map*:

Although collections like *List* and *Set* are completely opposite to each other, they have one common property. Both of them are value based collections.

Java provides another collection known as *Map* that stores elements in the form of key-value pairs where the key is unique but the values can be duplicates.

For every key, there is a corresponding value.

The *Map* interface has several implementation classes:

1. *Hashtable*
2. *HashMap*
3. *Properties*

Hashtable is thread-safe and does not accept *null*.

HashMap is not thread-safe and permits single *null* key but several *null* values.

It is possible to perform iterations over the *Map* using various options.

Fetching entries individually:

It is possible to fetch a single entry individually. Every entry contains a key and its corresponding value.

Java provides an inner interface *Map.Entry* that indicates an entry from the *Map*.

To obtain a set of type *Map.Entry*, *entrySet()* method is used.

Fetching values based upon keys:

In this case the entire set of keys is fetched from *Map* and then further iterations take place on that set.

This is possible using *keySet()* method.

To obtain a value from a *Map* bases upon its key, *get()* method is used.

Fetching values directly:

Sometimes it is necessary to fetch values directly irrespective of their keys.

This is possible using *values()* method.

Working with Iteration API:

The collections framework provides a separate API for performing iteration on the collections, known as Iteration API.

It mainly provides 3 interfaces:

1. *Iterator*
2. *ListIterator*
3. *Enumeration (Legacy Interface)*

Iterator:

It is used to perform iterations on the underlying collection.

To perform iterations, it is necessary to obtain an iterator on the top of the collection.

It is done by using *iterator()* method from *Collection* interface.

Once an *Iterator* is obtained, it can be used further by invoking its methods:

1. *hasNext()*
2. *next()*
3. *remove()*

The *hasNext()* method returns a *boolean* value indicating whether there is any entry available or not.

The *next()* method is used to obtain that entry and simultaneously the method shifts the cursor 1 step forward.

The *remove()* method is used to perform removal operation.

When invoked, removes the element from the underlying collection.

ListIterator:

It is extended from *Iterator*.

It allows to perform iterations in forward as well as backward direction.

It provides 2 methods: *hasPrevious()* and *previous()*.

In order to obtain *ListIterator* reference, *Collection* interface provides a method: *listIterator(position)*.

Enumeration:

It is a legacy interface and used to perform iterations in forward direction only.

It provides 2 methods:

1. *hasMoreElements()*
2. *nextElement()*

It is read-only whereas an *Iterator* provides *remove()* method that is used to perform removal.

Properties:

Java provides a separate class for *Map* implementation known as *Properties*.

It is meant for holding properties in the form of name-value pairs where every property has a name and its corresponding value.

This is especially used while working with configuration specific information.

E.g.

Every computer system has several properties e.g. OS Name, OS Version etc.

In order to access these properties, *Properties* class can be used.

It provides a method: *propertyNames()* that returns a list of property names in the form of *Enumeration*.

In order to retrieve a property value against its name, it provides a method: *String getProperty(String nameOfProperty)*.

While working with configuration information, generally the information is maintained in a separate file that is referred as *PROPERTIES* file.

This file has an extension: *.properties*.

It contains an information in the form of name-value pairs where every property name and its value are separated by '=' sign.

In order to load the information from *PROPERTIES* file, *Properties* class provides a *load()* method.

Using Collection as a Class Property:

A class can have a property declared of some collection type.

This is especially required when an object contains a property that may hold multiple values where the count is unknown.

Like regular fields, this property is also having getter and setter methods; however, in order to work with single value, it is always better to provide convenient methods.

Sorted Collections:

Sorting is a very common and frequent requirement in a business domain.

Java provides support for sorting through sorted collections.

There are 2 types of sorted collections:

1. *SortedSet*
2. *SortedMap*

SortedSet:

It is an extension to *Set*.

It does not accept duplicate values but stores elements in the sorted order.

It is implemented using *TreeSet*.

SortedMap:

It is an extension to *Map*.

It stores data in the form of key-value pairs but sorting is done on the basis of keys and not values.

It is implemented using *TreeMap*.

Even though all collections have a common characteristic that they can hold elements of dissimilar types unless they are declared type-safe, sorted collections is an exception to that.

They cannot hold elements of dissimilar types even though they are type unsafe because they arrange the elements in the sorted order and to achieve this behavior, elements must be of same type.

Working with User Defined Objects:

It is possible to store objects of a user defined class within a sorted collection.

However, in order to sort the elements using a sorted collection, the corresponding implementation class of the element has to implement an interface: *Comparable* from *java.lang* package.

All wrapper classes and *String* class already implement *Comparable* interface.

It provides a single method: *compareTo()* that is used to provide default sorting algorithm.

Customizing Sorting Algorithm:

The algorithm provided through *Comparable* interface implementation is the default sorting algorithm; however sometimes it is necessary to customize the sorting algorithm further.

This is made possible using one more interface: *Comparator* from *java.util* package.

It provides a method: *compare()*.

Day 8:

Utility Classes:

Java provides several classes meant for handling different types of functionalities and therefore these classes are referred as utility classes.

Most of these classes belong to *java.util* package.

Some of these classes are used to handle functionalities pertaining to collections.

E.g.

1. *Arrays*
2. *Collections*

Arrays:

It is used to perform operations on the top of arrays like searching, sorting etc.

It can also be used to populate a mutable *List* based upon some data values.

Collections:

It is used to work upon data present inside collections.

It provides several *static* methods to handle various functionalities.

Apart from these classes, there are several other utility classes available.

E.g.

1. *Date*
2. *Calendar*
3. *Scanner*
4. *StringTokenizer*
5. *Objects*
6. *Math (java.lang)*
7. *SimpleDateFormat (java.text)*

Other Building Blocks:

Java mainly provides 2 building blocks e.g. Classes and Interfaces.

However, apart from these, there are other several building blocks supported:

1. Annotations
2. Enums
3. Record Classes

Annotations:

Java version 5 introduced a feature known as Annotations.

Annotations are directly added into the source code for handling metadata.

They are denoted by '@' symbol e.g. `@Override`.

Annotation mainly has 2 properties:

1. Target
2. Retention Policy

Target:

It specifies the possible location where an annotation is to be applied.

A single annotation may have one or many possible targets.

These targets can be Type, Field, Method etc.

E.g. `@Override` annotation has only 1 possible target: Method.

Retention Policy:

It indicates at what time interval the annotation creates an impact.

There are 3 types of retention policies:

1. Source
2. Class
3. Runtime

An annotation is to be declared using *interface* keyword prefixed with '@' symbol.

An annotation may have one or many properties.

These properties are declared using method notations.

Enums:

It is a feature introduced by Java version 5.

It is used to declare variables of enumerated type.

It is used especially when a variable is supposed to hold a value that is one of the several possible values.

Enums are declared using *enum* keyword.

Like regular objects, properties of enum type can also be compared for equality using *equals()* method.

All enums have *equals()* method already implemented.

Record Classes:

Java version 16 introduced a feature Record Classes as preview and it has been finalized in version 17.

Record classes are used to make objects immutable.

When an object's content cannot be modified then that object is called as an immutable object.

Prior to version 16, to make objects immutable, a class used to define only getter methods but not setter methods.

However, this design had many complexities e.g. developer had to define each and every functionality like constructor, getter methods, *toString()*, *hashCode()* and *equals()*.

Record classes add simplicity to this design as they hide all these functionalities and handle them implicitly.

Java provides *record* keyword to create record classes.

Once a record or record class is defined, it by default provides implementation for constructor, *toString()*, getters, *hashCode()* and *equals()*.

Working with Reflection:

Sometimes, there is a requirement to fetch information about a class and take necessary action accordingly.

Java provides a special feature known as Reflection to achieve this.

At the core, Java provides a class named as *Class* from *java.lang* package.

The object of *Class* class holds information about some class e.g. name, constructors, fields, methods and so on.

In order to use *Class* class, it is necessary to obtain its reference which is possible using several options.

E.g. Loading the class explicitly using *static* method *forName()*.

This method loads the class explicitly and returns the reference of *Class* type.

It throws *ClassNotFoundException* if the relevant *.class* file of the class is not present.

In order to obtain other information like fields, methods, constructors, Java provides relevant classes from *java.lang.reflect* package:

1. *Field*
2. *Method*
3. *Constructor*

There is another approach available for obtaining the reference of *Class* class.

The topmost class *Object* provides a method: *getClass()* that returns a reference of *Class* class.

Working with NIO:

Java provides another way of handling Input and Output operations (R/W) through a feature NIO i.e. New IO.

The NIO functionality works almost similar to regular IO operations but can be used in some special cases e.g. making the calls asynchronous (Non-Blocking).

Java provides a main package *java.nio* that has several APIs including sub-packages e.g. *java.nio.file*.

It mainly consists of the following:

1. *Path*
2. *Paths*
3. *Files*

Path:

It is an interface used to configure the file path.

Paths:

It is a class used to obtain the reference of type *Path*.

Files:

A class used to perform R/W operations.

Working with Channels and Buffers:

The NIO functionality is well designed to work with channels and buffers.

A channel is a handle to the file whereas a buffer is responsible for storing the data.

Java provides relevant APIs to achieve this:

1. *FileChannel*
2. *ByteBuffer*

Java 8 Features:

Oracle Corporation launched a version 8 for Java in 2014 and it brought drastic enhancements in the programming model.

Therefore Java 8 features are widely used at industry level.

1. Interface Enhancements
2. Functional Interfaces
3. Method References
4. Lambda Expressions
5. Stream API
6. Date and Time API

Interface Enhancements (Reference: TestMain.java):

Till version 7, it was not possible to define methods within an interface.

Java 8 allows to define methods within an interface provided they are either *default* or *static*.

Functional Interfaces:

When an interface declares one and only one *abstract* method then that interface is called as Functional Interface.

E.g.

Runnable is a functional interface because it has a single *abstract* method: *run()*.

Java provides an annotation: *@FunctionalInterface* that is to be applied at the interface level so that a compiler can check whether that interface is a functional interface or not.

Method References (Reference: GreetMain.java, CountdownMain.java):

It is possible to create a reference to a method.

If the method's signature and the return type match with the same as that of the method declared in the functional interface, then the reference to such a method is compatible with the relevant functional interface type.

There are 2 types of method references:

1. Reference to a non-static method
2. Reference to a static method

In order to create a reference to a method, Java uses '::' notation.

In case of non-static method a class object is required whereas in case of static method it is not required.

Lambda Expressions:

Java 8 introduced a feature Lambda Expression to work with functional interfaces.

The lambda expression is divided into 3 parts:

1. Lambda Parameter
2. Lambda Operator
3. Lambda Body

Lambda Parameter:

It indicates the parameters to be used for the method from the functional interface for which the expression is defined.

It is denoted by '()'.

Lambda Operator:

It separates Lambda Body from Lambda Parameter.

It is denoted by '->'.

Lambda Body:

It is the actual implementation of the expression.

It can be specified using 2 options:

1. Single Statement
2. Block

If the method of a functional interface is parameterized, even though the type of the variable is not mentioned, it gets inferred.

If the method accepts a single parameter, the use of '()' is optional provided type is getting inferred.

If type is mentioned explicitly, '()' are mandatory.

If the lambda body contains a single statement and the method returns a value, *return* keyword must not be used.

If it contains multiple statements, then *return* keyword must be used.

Day 9:

Stream API:

Java 8 provides a feature Stream API that is used to perform several operations on the data held inside collections. Stream is a carrier of data. It simply moves the data rather than storing it.

Prior to Java 8, data processing on collection was time consuming.

Due to Stream API and functional programming, it becomes very convenient.

Stream API can be used to handle various operations like:

1. Iterate
2. Filter
3. Map
4. Reduce
5. Collect

Iterate:

Used to perform iterations.

Filter:

Used to filter the results based upon some condition

Map:

Used to transform a collection of one type into another

Reduce:

Used to process the data to get a single value

Collect:

Used to transform a Stream into some Collection.

Working with Stream API:

In order to work with Streams, it is necessary to use stream specific library that belongs to a package: `java.util.stream`.

It provides several predefined classes and interfaces but at the base there is an interface called as `Stream`.

It is the main resource responsible for handling stream operations and therefore to proceed with these operations, it is necessary to obtain its reference.

All collection specific APIs have been modified since version 8 in such a way that they provide a method: `stream()` that returns a reference of type: `Stream`.

Once a reference is obtained, it can be used to perform further operations.

This is done using relevant methods of `Stream` interface.

E.g.

To perform iterations, `forEach()` method is used.

Most of these methods accept a parameter of some interface type and that interface is a functional interface.

E.g.

The `forEach()` method accepts a reference of type `Consumer` which is a functional interface.

Since it is a functional interface, it can be handled using either method references or lambda expressions.

The `forEach()` method is used to perform iterations.

Filter:

It is used to filter out the results based upon some condition.

The *Stream* interface provides *filter()* method to achieve this.

The *filter()* method accepts a reference of functional interface type called as *Predicate*, uses its logic for filtration and produces a new stream.

Map:

It is used to transform stream of one type into another one.

The *Stream* interface provides *map()* method to achieve this.

The method accepts a parameter of functional interface type: *Function*.

Reduce:

It is used to perform operation on the data to get a single value.

E.g. Addition of numbers available in the collection.

The *Stream* interface provides *reduce()* method to achieve this.

It accepts a reference of functional interface type: *BinaryOperator* and returns a value of type *Optional*.

Java 8 provides a class named as *Optional* which is meant for either holding a value or not.

In some cases, it might happen that after processing the functionality, the value to be returned may exist or not.

If it does not exist, ideally *null* value is returned but it may cause *NullPointerException*.

To address this problem, *Optional* class has been introduced.

Collect:

The stream populated on some underlying collection is used to perform various operations like filter, map, reduce and so on.

However, the stream itself does not store the data; rather it only carries the data.

Sometimes it is necessary to populate a collection based upon the results generated due to stream operations.

This is done with the help of collect operation.

The *Stream* interface provides *collect()* method to achieve this.

It accepts a reference of type *Collector* and it is not a functional interface and therefore cannot be used with method reference or even a lambda expression.

Java 8 provides a class: *Collectors* that provides several methods through which a reference of type *Collector* can be populated.

Date and Time API:

Java 8 provides an API called as Date and Time API that is used to handle the functionalities about date and time.

Prior to Java 8, the date and time specific functionalities were getting handled by classes like *Date*, *Calendar* etc.

Even though these classes are capable of handling date and time, they are not intuitive.

E.g. the year starts with 1900

To address these problems, a separate date/time API has been provided.

Java supports Date/Time API through a package: *java.time*.

It mainly consists of 3 classes:

1. *LocalDate*
2. *LocalTime*
3. *LocalDateTime*

All these classes are to be instantiated through their respective *static* methods.

Java API Enhancements:

Since version 10, Oracle Corporation changed the policy to time-based versioning.

A new version gets launched for every 6 months.

Java introduced several enhancements through its versions.

Private methods within interface:

Java version 9 introduced a feature of *private* methods defined within an interface.

Local Variable Type Inference:

It is feature introduced by Java version 10.

When a variable declared is a local variable, it is possible to get the type of that variable inferred.

Java provides **var** keyword to achieve this.

However, if the local variable is defined as a parameter in method signature, the *var* keyword cannot be used.

Modified *switch* expressions:

A feature introduced by Java 14.

It allows to use the case expressions without *break* statement.

A regular *switch...case* construct always needs *break* statement to be used otherwise; once the match is found, remaining cases also execute until the next *break*.

Apart from this, the modified *switch* expression uses '*->*' notation instead of '*:*'.

It is also possible to assign a *switch* expression to a variable.

If the expression contains a single statement, only the value is to be returned.

If the expression contains multiple statements, the value is to be returned using *yield* keyword and not the *return* keyword.

Text Blocks:

It is a feature introduced by Java 15.

It allows to create a block holding multiple lines.

Prior to this feature, in order to add a new line, '*\n*' was mandatory.

Text blocks simplify this requirement and they are denoted by [*"""*] at the start and end.

Pattern Matching for *instanceof* keyword:

It is a feature introduced by Java 16 that allows to check the condition using *instanceof* keyword and once the match is found, store the reference directly into a variable.

Sealed Classes:

A feature introduced by Java 17.

When a class is declared as *abstract*, it cannot be instantiated and it is to be used further by creating its subclass.

When a class is declared as *final*, it cannot be extended by any subclass.

Therefore a class can be declared as either *abstract* or *final* but not both.

Sealed classes are used to provide the functionality between *abstract* and *final*.

In order to use sealed classes, Java provides several keywords:

1. *sealed*
2. *permits*
3. *non-sealed*

When a class is declared as *sealed*, it must provide permissions to the classes for further extension.
This is done using *permits* keyword.

When a subclass extends a super class that is a sealed class, the subclass must be declared either *non-sealed* so that any class can further extend it, *final* so that no class can further extend it or even *sealed* to provide further permissions.

=====THE END=====